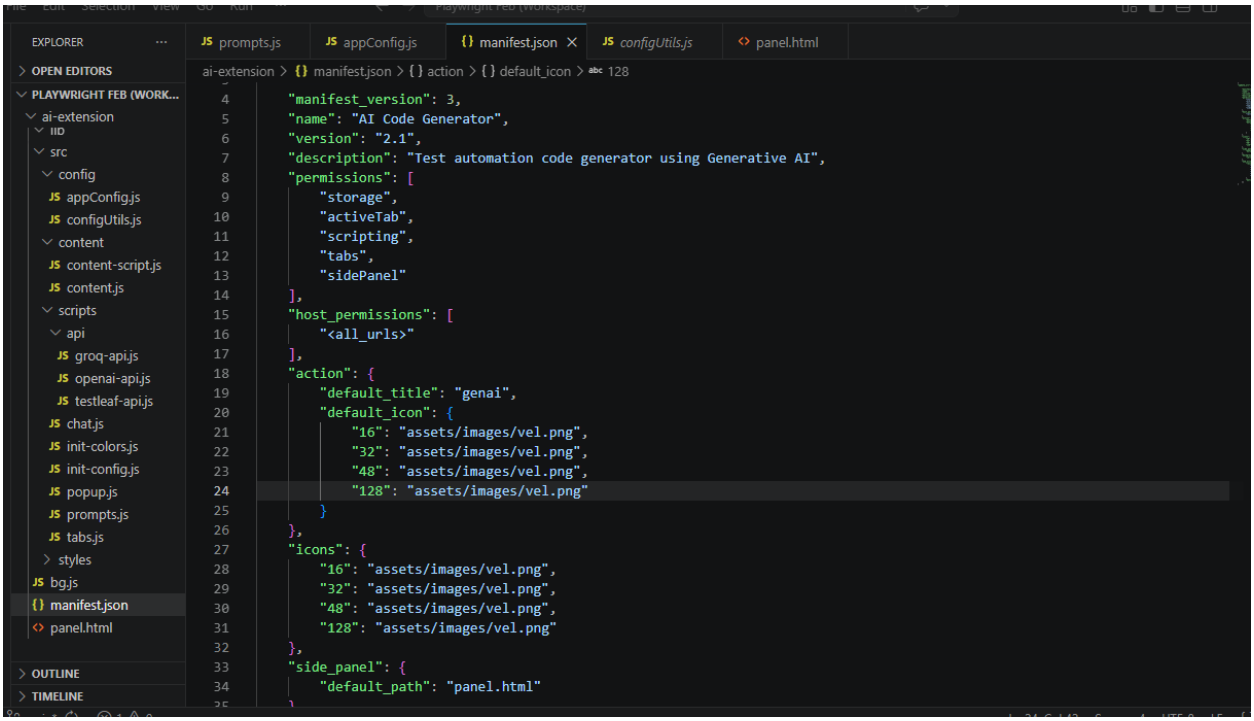
```
/**

Added the code in the prompt.js file

   * Playwright TypeScript Page/Test Prompt
   */
  PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY: `
    Instructions:
    - Generate ONLY a Playwright TypeScript test file using @playwright/test.
    - Use async/await and page.locator for selectors.
    - Do NOT include any explanation, comments outside the code, or non-
TypeScript content.
    - Keep the code executable and compatible with Playwright Test.
    - Use Playwright weblocators(attributes and then prefer xpath)
    - use meaning ful method name
    - Prepare a readme file atlast
    - implement a ts and js file as needed


    Context:
    DOM:
    \`\`\`html
    \${domContent}
    \`\`\`
    URL: \${pageUrl}

    Example:
    \`\`\`ts
    import { test, expect } from '@playwright/test';

    test('should log in successfully', async ({ page }) => {
      await page.goto('\${pageUrl}');
      await page.locator('input[name="username"]').fill('testuser');
      await page.locator('input[name="password"]').fill('password123');
      await page.locator('button[type="submit"]').click();
      await expect(page).toHaveURL(/dashboard/);
    });
    \`\`\`

    Persona:
    - Audience: automation engineers using Playwright with TypeScript.

    Output Format:
    - A single TypeScript file inside a \`\`\`ts\`\`\` block.
```

```
    Tone:
    - Clean, maintainable, executable.
  `,
```

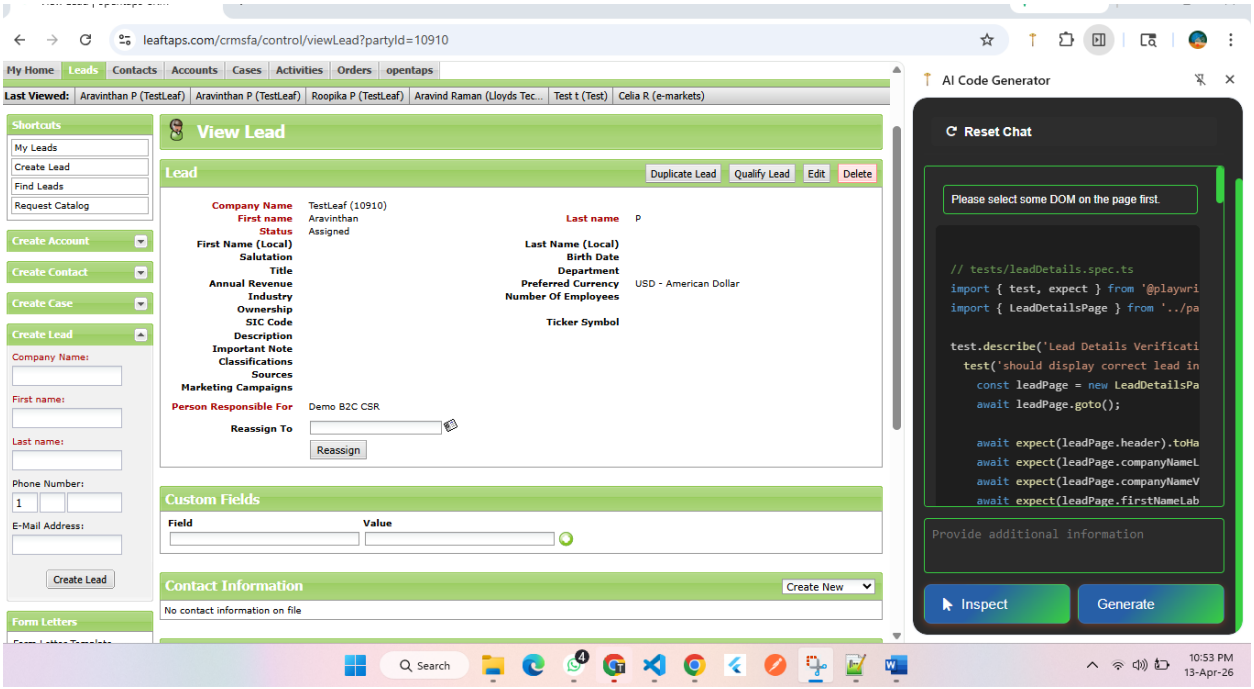
Updated the png image as well in Manifest and appconfig.js file



Reloaded the extension in the chrome

Logged in the leafaps.com

Select the lead -> modify




```
// leadViewPage.ts
import { Page, Locator } from '@playwright/test';

export class LeadViewPage {
  readonly page: Page;
  readonly companyName: Locator;
  readonly firstName: Locator;
  readonly status: Locator;
  readonly reassignInput: Locator;
  readonly reassignButton: Locator;
  readonly contactMechSelect: Locator;

  constructor(page: Page) {
    this.page = page;
    this.companyName = page.locator("//span[@id='viewLead_companyName_sp']");
    this.firstName = page.locator("//span[@id='viewLead_firstName_sp']");
    this.status = page.locator("//span[@id='viewLead_statusId_sp']");
    this.reassignInput =
page.locator("//input[@id='reassignToForm_newPartyId']");
    this.reassignButton = page.locator("//input[@id='ext-gen643']");
    this.contactMechSelect =
page.locator("//select[@id='createNewContactMechTarget']");
  }

  async navigateTo(leadId: string) {
    await
this.page.goto(`https://leaftaps.com/crmsfa/control/viewLead?partyId=${leadId
}`);
  }
```

```
  async getCompanyName(): Promise<string> {
    return await this.companyName.textContent() ?? '';
  }

  async getFirstName(): Promise<string> {
    return await this.firstName.textContent() ?? '';
  }

  async getStatus(): Promise<string> {
    return await this.status.textContent() ?? '';
  }

  async reassignTo(partyId: string) {
    await this.reassignInput.fill(partyId);
    await this.reassignButton.click();
  }

  async selectContactMech(optionValue: string) {
    await this.contactMechSelect.selectOption({ value: optionValue });
  }
}

// leadView.spec.ts
import { test, expect } from '@playwright/test';
import { LeadViewPage } from './leadViewPage';
import { generateRandomId } from './utils';

test.describe('Lead View Page Tests', () => {
  test('verify lead details and perform reassign', async ({ page }) => {
    test.info().annotations.push(
      { type: 'Author', description: 'Automation Engineer' },
      { type: 'TestCase', description: 'Verify lead details and reassign
responsibility' }
    );

    const leadId = '10910';
    const leadPage = new LeadViewPage(page);
    await leadPage.navigateTo(leadId);

    await expect(leadPage.companyName).toBeVisible();
    await expect(leadPage.firstName).toBeVisible();
    await expect(leadPage.status).toBeVisible();

    const companyName = await leadPage.getCompanyName();
    const firstName = await leadPage.getFirstName();
    const status = await leadPage.getStatus();

    expect(companyName).toContain('TestLeaf');
    expect(firstName).toBe('Aravinthan');
    expect(status).toBe('Assigned');

    const newPartyId = generateRandomId();
    await leadPage.reassignTo(newPartyId);
    // Assuming a success toast appears; adjust selector as needed
    const successToast = page.locator("//div[contains(@class,'toast') and
contains(.,'Reassigned')]");
```

```
    await expect(successToast).toBeVisible();

    await
leadPage.selectContactMech('/crmsfa/control/leadsEditContactMech?partyId=1091
0&preContactMechTypeId=POSTAL_ADDRESS&DONE_PAGE=viewLead?partyId%3d10910');
    // Verify selection
    await
expect(leadPage.contactMechSelect).toHaveValue('/crmsfa/control/leadsEditCont
actMech?partyId=10910&preContactMechTypeId=POSTAL_ADDRESS&DONE_PAGE=viewLead?
partyId%3d10910');
  });
});

// utils.js
function generateRandomId(length = 8) {
  const chars =
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789';
  let result = '';
  for (let i = 0; i < length; i++) {
    result += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  return result;
}
module.exports = { generateRandomId };

# README.md

## Project Structure

## Prerequisites
- Node.js (>= 18)
- Playwright installed (`npm i -D @playwright/test`)
- Typescript configured (`tsconfig.json`)

## Setup
1. Install dependencies


2. Ensure the `utils.js` file is accessible from the test (CommonJS export is
used).

## Running the Test

The test will:
1. Navigate to the lead view page for lead ID `10910`.
2. Verify the displayed company name, first name, and status.
3. Reassign the lead to a randomly generated party ID.
4. Select the "Address" option from the contact mechanism dropdown.

## Notes
- Adjust selectors or URLs if the application UI changes.
- The success toast selector in the test may need to be updated based on
actual implementation.
```
Copy